Conversely, the code in a method definition is executed later, when you eventually call the method. However, as you write your program, you usually don't care *when* it changes scope—you only care that it does.

Now you can pinpoint the places where your program changes scope—the spots marked by class, module, and def. But what if you want to pass a variable through one of these spots? This question takes you back to blocks.

## Flattening the Scope

The more you become proficient in Ruby, the more you get into difficult situations where you want to pass bindings through a *Scope Gate (81)*:

blocks/flat_scope_1.rb

```
my_var = "Success"

class MyClass
  # We want to print my_var here...
  def my_method
    #  ..and here
  end
end
```

Scope Gates are quite a formidable barrier. As soon as you walk through one of them, local variables fall out of scope. So, how can you carry my_var across not one but two Scope Gates?

Look at the class Scope Gate first. You can't pass my_var through it, but you can replace class with something else that is not a Scope Gate: a method call. If you could call a method instead of using the class keyword, you could capture my_var in a closure and pass that closure to the method. Can you think of a method that does the same thing that class does?

If you look at Ruby's documentation, you'll find the answer: Class.new is a perfect replacement for class. You can also define instance methods in the class if you pass a block to Class.new:

blocks/flat_scope_2.rb

```
my_var = "Success"

➤ MyClass = Class.new do
➤   # Now we can print my_var here...
➤   puts "#{my_var} in the class definition!"

    def my_method
      # ...but how can we print it here?
    end
end
```